

Pattern Matching and Data — A Readable Companion to the Elixir Docs

Chapter 1 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why pattern matching is the core of Elixir

In most languages, `=` assigns. In Elixir it *matches*: the left side is a pattern, and the match either succeeds, binding variables, or raises. That one idea runs through the whole language: function heads, `case`, `with`, `receive`, and the way you are expected to handle data you don't trust.

It also sets the style. Elixir code is meant to be *assertive*: state the shape you expect and let anything else fail immediately, close to its cause. The official anti-pattern guide lists the opposite, *non-assertive* code, as a named anti-pattern three times over. Much plausible-but-wrong Elixir is defensive code written with habits from other languages: returning `nil` "to be safe", reading a map with `[]` "in case the key is missing", accepting anything truthy.

2. The building blocks

Matching

Construct	Meaning
<code>{:ok, v} = f()</code>	Match or raise <code>MatchError</code> ; binds <code>v</code>
<code>^x</code>	Compare with the existing value of <code>x</code> instead of rebinding
<code>def f(%{id: id})</code>	Match in a function head; next clause if it fails
<code>when guard</code>	Extra condition on a clause, from a restricted set of expressions
<code>_</code> / <code>_name</code>	Match anything without binding (or bind and mark unused)

Guards allow only a fixed set of expressions: comparisons, `and` / `or` / `not`, arithmetic, type checks like `is_map/1`, and some built-ins like `map_size/1`. `&&`, `||` and `!` are not allowed, because they are not strictly boolean.

Choosing a data structure

Structure	Keys	Order	Duplicates	Access	Use for
Map <code>%{}</code>	Any term	Not guaranteed	No	Logarithmic time	General key-value data

Structure	Keys	Order	Duplicates	Access	Use for
Struct <code>%User{}</code>	Fixed atom set	—	No	<code>user.field</code>	Domain data with a known shape
Keyword list <code>[a: 1]</code>	Atoms	Kept	Allowed	Linear scan	Options passed to functions
Tuple <code>{:ok, v}</code>	Positional	Fixed	—	By pattern	Small fixed groupings, tagged results

A struct is a map with an extra `__struct__` field. It does not implement `Access` (so no `user[:name]`) or `Enumerable`.

Two ways to read a map

The docs draw the line clearly: `map.key` is for maps with “a predetermined set of atom keys, which are expected to always be present,” and raises `KeyError` if one is missing. `map[key]` is for dynamic maps that “may have any key,” and returns `nil` when it is absent.

3. How it actually works

Guards fail, they don’t raise

The docs: “In guards, when functions would normally raise exceptions, they cause the guard to fail instead.” A guard calling `map_size/1` on a list does not crash; the whole guard is simply false, and the next clause is tried. That is useful, and it is also the source of a subtle bug (section 4.2).

Truthiness

Only `false` and `nil` are falsy. Everything else is truthy, including `0`, `""`, `[]` and the string `"false"`. `&&`, `||` and `if` use truthiness; `and`, `or` and `not` require actual booleans and raise otherwise.

@enforce_keys is a construction check

`@enforce_keys` makes `%User{}` raise if a key is missing at that moment. The docs note it “is not enforced on updates and it does not provide any sort of value-validation.” Functions that build structs from data, like `struct/2`, don’t check it either; `struct!/2` does.

Big structs change representation

The runtime stores maps with up to 32 keys compactly. Above that it switches to a different internal structure, which the anti-pattern guide warns can lead to “bloating and higher memory usage” for structs; every instance of a large struct pays it.

4. Plausible but wrong

4.1 Reading required fields with []

```
def send_welcome(user) do
  Mailer.deliver(to: user[:email], name: user[:name])
end
```

If `:email` is missing, because of a typo in the key or an upstream change, this passes `nil` to the mailer, which fails somewhere far away with a confusing error. Or worse, it succeeds and sends nothing. Fix: `user.email` or a struct pattern (`%User{email: email}`), so a missing field raises `KeyError` right here. Keep `[]` for maps whose keys really are optional or dynamic.

4.2 An `or` guard that silently fails

```
def normalize(x) when map_size(x) > 0 or is_list(x), do: :ok
def normalize(_), do: :other

normalize([1])  #=> :other
```

For a list, `map_size/1` fails, which fails the *entire* guard, so the `is_list` half never counts. The code looks as if it accepts lists; it never does. Fix: separate guard clauses, each safe on its own:

```
def normalize(x) when is_map(x) and map_size(x) > 0, do: :ok
def normalize(x) when is_list(x), do: :ok
```

4.3 A variable that should have been pinned

```
expected = :ok
case do_work() do
  expected -> :success      # rebinds `expected`, matches anything
  _        -> :failure
end
```

Without `^`, `expected` in the pattern is a fresh variable that matches every value, so the function reports success for `:error` too. The only clue is a compiler warning about an unused variable. Fix: `^expected -> :success`, and treat that warning as an error.

4.4 Defensive code that hides bad data

```
def parse_pair(s) do
  case String.split(s, "=") do
    [k, v] -> {k, v}
    _      -> nil
  end
end
```

Malformed input becomes `nil`, which travels on and blows up in another module, long after the input that caused it. The anti-pattern guide calls this non-assertive pattern matching. Fix: if bad input is a bug, assert: `[k, v] = String.split(s, "=", parts: 2)`. If it is expected, return `{:error, :invalid_pair}` so the caller has to handle it (chapter 2).

4.5 Trusting `@enforce_keys`

```
defmodule User do
  @enforce_keys [:email]
  defstruct [:email, :name]
end

user = struct(User, params) # params has no :email
```

`struct/2` does not check enforced keys, so this builds `%User{email: nil}` without complaint. The same happens with `%{user | email: nil}` or `Map.put`. Fix: build with `%User{...}` or `struct!/2`, and validate values at the boundary (a changeset or an explicit `new/1` that returns `{:ok, user} | {:error, reason}`).

4.6 Truthy strings from the environment

```
if System.get_env("FEATURE_X") do
  enable_feature_x()
end
```

With `FEATURE_X=false`, `System.get_env/1` returns the string `"false"`, which is truthy, so the feature turns on. The same happens with `"0"` or `"no"`. Fix: parse to a real boolean once, at startup, in `config/runtime.exs` (for example `System.get_env("FEATURE_X") == "true"`), and compare with `==` rather than relying on truthiness.

4.7 Defaults that override options

```
def connect(opts) do
  opts = @defaults ++ opts
  timeout = Keyword.get(opts, :timeout)
```

Keyword lists allow duplicate keys, and `Keyword.get/2` returns the *first*. With the defaults first, the caller's `timeout: 100` is ignored and the default of 5000 wins. Fix: `Keyword.merge(@defaults, opts)`, where the second list's values win, or `Keyword.validate!/2` (Elixir 1.13+), which also rejects unknown options.

4.8 A struct that grows past 32 fields

```
defmodule Order do
  defstruct [:id, :customer_id, :status, ...] # 40 fields
end
```

Above 32 fields each instance uses the larger map representation, and with millions of orders in memory or in ETS the cost adds up. It usually also means the struct holds several concepts at once. Fix: group

related fields into nested structs (`%Order{shipping: %Address{}, totals: %Totals{}}`).

5. Review checklist and sources

- ❑ Required fields are read with `map.key`, pattern matching or structs, not `map[:key]`
- ❑ Guards that combine checks are safe for every input type, or split into separate clauses
- ❑ Values being compared in patterns are pinned with `^`; unused-variable warnings are errors in CI
- ❑ Unexpected data either fails fast or returns `{:error, reason}`; never a silent `nil`
- ❑ Structs from external data go through `struct!/2` or a validating constructor
- ❑ Environment strings are parsed to real types once, at startup
- ❑ Options are combined with `Keyword.merge/2` or checked with `Keyword.validate!/2`
- ❑ Structs stay under 32 fields

Sources

- Patterns and guards: <https://hexdocs.pm/elixir/patterns-and-guards.html>
- Map: <https://hexdocs.pm/elixir/Map.html>
- Keyword: <https://hexdocs.pm/elixir/Keyword.html>
- Structs: <https://hexdocs.pm/elixir/structs.html>
- Code-related anti-patterns: <https://hexdocs.pm/elixir/code-anti-patterns.html>